

MEDIUM

Mutable Default Argument Trace

A teammate wrote the helper function below to collect items into a running list. Read it carefully and trace what each call actually does to the underlying list object, keeping in mind how Python evaluates default argument values.

```
def append_item(item, bucket=[]):
    bucket.append(item)
    return bucket

print(append_item(1))
print(append_item(2))
print(append_item(3, []))
print(append_item(4))
```

Your task:

1. Write the exact text printed by each of the four print statements, in order.
2. State whether the calls `append_item(1)`, `append_item(2)`, and `append_item(4)` share the same underlying list object, and how you can tell.
3. Explain in 1-2 sentences why the default argument `bucket=[]` is evaluated only once, not on every call.

MEDIUM

Late-Binding Closures In A Loop

The snippet below builds a list of no-argument lambdas inside a loop, intending to capture each loop value separately. Trace how Python actually resolves the free variable `i` inside each lambda.

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)

results = [f() for f in funcs]
print(results)
```

Your task:

1. Write the exact value printed for results.
2. Explain why all three lambdas do not return their 'own' loop value (0, 1, 2 respectively).
3. Describe one minimal change to the lambda definition that would make each closure capture its own value of i at creation time.

MEDIUM

Decorator Application Order

Two decorators are stacked on the greet function below. Trace precisely which decorator wraps which, and in what order the wrapped behavior is applied when greet is called.

```
def shout(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper

def exclaim(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs) + "!"
    return wrapper

@exclaim
@shout
def greet(name):
    return f"hello {name}"

print(greet("sam"))
```

Your task:

1. Write the exact string printed.
2. State which decorator is applied to the original greet function first, and which one wraps the result of that.
3. State whether swapping the order of the two @ lines (@shout above @exclaim) would change the final printed output, and justify your answer either way.

MEDIUM

String Slicing Edge Cases

The snippet below performs several slicing operations on the same string. Trace each one carefully, paying attention to step values and out-of-range bounds.

```
s = "engineering"
print(s[::-1])
print(s[2:8:2])
print(s[-4:])
print(s[100:])
```

Your task:

1. Write the exact string printed by each of the four print statements, in order.
2. Explain why `s[100:]` does not raise an `IndexError` even though 100 is far beyond the length of the string.

MEDIUM

Generator Exhaustion Behavior

The function below returns a generator object. Trace what happens when the same generator instance is consumed more than once, versus creating a fresh generator each time.

```
def counter(n):
    for i in range(n):
        yield i

gen = counter(3)
print(list(gen))
print(list(gen))
print(sum(counter(3)))
```

Your task:

1. Write the exact value printed by each of the three print statements, in order.
2. Explain why the second print statement does not repeat the values from the first.
3. Explain why the third print statement behaves differently from the first two.

MEDIUM

Leaky Shared Default Dictionary

A utility groups words by their length so a caller can process one batch of words at a time. In testing, unrelated calls appear to affect each other's results: calling the function with a fresh, unrelated batch of words still shows entries left over from a previous call.

```
def group_by_length(words, groups={}):
    for w in words:
        groups.setdefault(len(w), []).append(w)
    return groups

print(group_by_length(["cat", "dog"]))
print(group_by_length(["ox"]))
```

Your task:

1. Explain exactly why the second call's result includes data left over from the first call.
2. Rewrite `group_by_length` so each call without an explicit `groups` argument starts from an empty dictionary.
3. Make sure a caller can still optionally pass in their own dictionary to accumulate into, if they want that behavior.

MEDIUM

Off-By-One Adjacent Comparison

The function below is supposed to check whether a list of numbers is sorted in non-decreasing order by comparing each element to its neighbor. It crashes on valid, well-formed input.

```
def is_sorted_ascending(nums):
    for i in range(len(nums)):
        if nums[i] > nums[i + 1]:
            return False
    return True

print(is_sorted_ascending([1, 2, 3]))
```

Your task:

1. State exactly which line raises an error, what kind of error it is, and for which value of `i`.
2. Write a corrected version of `is_sorted_ascending` that works for lists of any length, including empty lists and single-element lists.

MEDIUM

Unbound Local Total Accumulator

A module-level total is meant to be incremented by a helper function each time it's called. Calling the function raises an error before it can even run the addition.

```
total = 0

def add_to_total(x):
    total += x
    return total

print(add_to_total(5))
```

Your task:

1. State the exact exception raised and explain, in terms of Python's scoping rules, why it happens even though `total` is defined at module level.
2. Redesign the code to avoid relying on mutable module-level state altogether, using a closure with `nonlocal` to keep a private running `total` instead.
3. Show two calls to your redesigned accumulator that demonstrate the `total` increasing correctly across calls.

MEDIUM

Missing Base Case In Recursive Flatten

The function below is meant to recursively sum all numbers in an arbitrarily nested list of lists. It works for some inputs but crashes for others.

```
def flatten_sum(data):
    if isinstance(data[0], list):
        return flatten_sum(data[0]) + flatten_sum(data[1:])
    else:
        return data[0] + flatten_sum(data[1:])

print(flatten_sum([1, [2, 3], [4, [5, 6]]]))
```

Your task:

1. Explain exactly why this raises an error, and identify which recursive call first receives the problematic input.
2. Write a corrected version of `flatten_sum` that handles this case and still returns the correct total for nested input.
3. State what `flatten_sum([])` should return in your corrected version, and why.

MEDIUM

Deleting From A Dict While Iterating

An inventory cleanup function is supposed to remove expired items from a dictionary keyed by item id. It raises an error partway through, even though the dictionary itself looks fine before the call.

```
def remove_expired(inventory):
    for key, value in inventory.items():
        if value["expired"]:
            del inventory[key]
    return inventory

inv = {"a": {"expired": True}, "b": {"expired": False}, "c": {"expired": True}}
print(remove_expired(inv))
```

Your task:

1. State the exact exception raised and explain why mutating a dictionary while iterating over it directly causes this.
2. Write a corrected version of `remove_expired` that safely removes all expired entries.
3. Show the corrected function's output for the given `inv` dictionary.

MEDIUM

Why Generators Save Memory

The function below reads a large text file line by line using a generator instead of returning a list of all lines. Explain the reasoning behind this design choice.

```
def read_large_file_lines(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()
```

Your task:

1. Explain in your own words why using `yield` here uses far less memory than reading the whole file into a list with `f.readlines()`, especially for very large files.
2. Explain what happens if a caller tries to iterate over the same generator object returned by `read_large_file_lines` a second time.
3. Name one downside of using a generator here instead of returning a list, from the caller's perspective.

MEDIUM

Set-Based Duplicate Detection

The function below finds which items appear more than once in a list, using two sets instead of, say, counting occurrences with a list and nested loops.

```
def find_duplicates(items):
    seen = set()
    duplicates = set()
    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return duplicates
```

Your task:

1. Explain why using sets here is more efficient than checking membership against a plain list, in terms of time complexity.
2. Explain why the function returns a set of duplicates rather than a list -- what property of the output does that guarantee (or not guarantee)?
3. State what would go wrong, or what would need to change, if items could contain unhashable elements like lists.

MEDIUM

How lru_cache Speeds Up Fibonacci

The naive recursive Fibonacci function below is decorated with `functools.lru_cache`. Explain what this decorator changes about how the function executes.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

Your task:

1. Explain how `lru_cache` changes the time complexity of computing `fib(n)` compared to the same function without the decorator.
2. Explain, in terms of hashing, why `lru_cache` requires its arguments to be hashable, and give an example of a call that would fail because of this.
3. Explain what `maxsize=None` means here, and describe one risk of using an unbounded cache in a long-running process.

PYTHON

MEDIUM

Context Manager Exception Semantics

The Timer class below implements the context manager protocol so it can be used with a with statement to measure elapsed time. Explain how the protocol works and what its return value from `__exit__` means.

```
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        return False
```

Your task:

1. Explain when `__enter__` and `__exit__` are each called relative to the body of a `with` block.
2. Explain what the three parameters `exc_type`, `exc_val`, `exc_tb` represent, and under what condition they are all `None`.
3. Explain what would change in caller behavior if `__exit__` returned `True` instead of `False` when an exception occurs inside the `with` block.

MEDIUM

Recursion Depth And Stack Cost

The recursive factorial function below works correctly for small inputs. Explain its resource usage and its limitations for large inputs.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

Your task:

1. Explain, in terms of the call stack, what happens as `factorial(n)` recurses for increasingly large `n`.
2. Explain why calling `factorial(10000)` is likely to fail in standard CPython, and name the specific exception involved.
3. Describe how you would rewrite this function to avoid that limitation, without changing its observable input/output behavior.

PYTHON

MEDIUM

Loop To List Comprehension

The function below filters a list of numbers down to the even ones and squares each one, using an explicit accumulator loop.

```
def even_squares(nums):
    result = []
    for n in nums:
        if n % 2 == 0:
            result.append(n ** 2)
    return result
```

Your task:

1. Rewrite `even_squares` to use a single list comprehension instead of an explicit loop and accumulator list, producing identical output.
2. State one situation where the original loop-based version might still be preferable to the comprehension, such as readability, debugging, or side effects.

MEDIUM

Recursive Fibonacci To Iterative

The function below computes the nth Fibonacci number using naive recursion, which recomputes the same subproblems many times.

```
def fib_recursive(n):
    if n < 2:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)
```

Your task:

1. Rewrite this as an iterative function `fib_iterative(n)` that computes the same values using a loop and a constant number of variables, without recursion.
2. State the time complexity of your iterative version versus the original recursive version.

PYTHON

MEDIUM

Iterator Class To Generator Function

The class below implements Python's iterator protocol manually with `__iter__` and `__next__` to lazily produce Fibonacci numbers up to a given count.

```
class FibonacciIterator:
    def __init__(self, limit):
        self.limit = limit
        self.a, self.b = 0, 1
        self.count = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.limit:
            raise StopIteration
        value = self.a
        self.a, self.b = self.b, self.a + self.b
        self.count += 1
        return value
```

Your task:

1. Rewrite this as a single generator function `fibonacci_gen(limit)` using `yield`, producing the exact same sequence of values when iterated.
2. Explain what state the class manages explicitly as instance attributes that the generator function manages implicitly instead.

MEDIUM

Nested Loops To `itertools.product`

The function below builds every possible pair combining one element from list1 with one element from list2, using two nested for loops.

```
def all_pairs(list1, list2):
    pairs = []
    for a in list1:
        for b in list2:
            pairs.append((a, b))
    return pairs
```

Your task:

1. Rewrite `all_pairs` to use `itertools.product` instead of nested loops, returning a list of tuples in the same order as the original.
2. Explain briefly why `itertools.product` scales better than manually nested loops when combining three or more input lists instead of two.

PYTHON

MEDIUM

Manual String Concatenation To Join

The function below builds a single comma-separated CSV row from a list of field values, using string concatenation with += inside a loop and manual index checks.

```
def build_csv_row(fields):
    row = ""
    for i, field in enumerate(fields):
        row += str(field)
        if i != len(fields) - 1:
            row += ","
    return row
```

Your task:

1. Rewrite build_csv_row using str.join and a comprehension or generator expression instead of a loop with += and an index check.
2. Explain briefly why repeated += concatenation in a loop is less efficient than str.join for building a large string, in terms of what happens to the string in memory on each +=.

MEDIUM

Top-K Words With Counter

You need a small utility that, given a block of text, returns the k most frequently occurring words along with their counts.

```
def top_k_words(text, k):
    # TODO: implement using collections.Counter
    pass
```

Your task:

1. Implement `top_k_words(text, k)` so it lowercases the text, splits it on whitespace into words, and returns a list of (word, count) tuples for the k most common words, most frequent first.
2. Use `collections.Counter` and its `most_common` method rather than manually counting with a dictionary and sorting.

MEDIUM

Grouping Records With defaultdict

You have a list of (name, grade) tuples and need to group student names by their grade letter, without manually checking whether each key already exists in the result dictionary.

```
def group_students_by_grade(students):
    # TODO: implement using collections.defaultdict
    pass
```

Your task:

1. Implement `group_students_by_grade(students)` so it returns a regular dict mapping each grade letter to a list of student names with that grade, preserving the order names were first seen.
2. Use `collections.defaultdict(list)` to avoid explicit 'if key not in dict' checks.

MEDIUM

Run-Length Encoding With groupby

You need to compress a string by collapsing consecutive runs of the same character into (character, count) pairs, classic run-length encoding.

```
def run_length_encode(s):
    # TODO: implement using itertools.groupby
    pass
```

Your task:

1. Implement `run_length_encode(s)` using `itertools.groupby` so that `run_length_encode('aaabbbcccd')` returns `[('a', 3), ('b', 3), ('c', 2), ('d', 1)]`.
2. Explain briefly why `itertools.groupby` only groups consecutive equal elements, and why that is exactly the right tool here without needing to sort the input first.

MEDIUM

Function Composition With reduce

You want a `compose(*funcs)` helper that takes any number of single-argument functions and returns one function which applies them left to right, so the output of one becomes the input of the next.

```
def compose(*funcs):
    # TODO: implement using functools.reduce
    pass
```

Your task:

1. Implement `compose(*funcs)` using `functools.reduce` so that `compose(f, g, h)(x)` is equivalent to `h(g(f(x)))`, meaning `funcs` are applied in the order given, left to right.
2. Demonstrate your implementation with `compose(lambda x: x + 1, lambda x: x * 2, str)(3)`, and state what it returns.

PYTHON

MEDIUM

Validated Dataclass With Post Init

You need a small Product record type with a name, a price, and an optional list of tags, where a negative price should be rejected immediately when a Product is constructed.

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class Product:
    name: str
    price: float
    tags: List[str] = field(default_factory=list)

    # TODO: add validation
```

Your task:

1. Add a `__post_init__` method to Product that raises `ValueError` if price is negative.
2. Explain briefly why tags uses `field(default_factory=list)` instead of writing `tags: List[str] = []` directly.
3. Demonstrate that two separately constructed Product instances do not share the same underlying tags list.

PYTHON

MEDIUM

Private State Via Closures

You want a `make_counter()` factory function that returns an increment function. Each call to `make_counter()` should produce an independent counter with its own private running total that can't be accessed or modified except by calling the returned function.

```
def make_counter(start=0):
    # TODO: implement using a closure
    pass
```

Your task:

1. Implement `make_counter(start=0)` so it returns a function `increment(step=1)` that adds `step` to an internal count, beginning at `start`, and returns the new count.
2. Demonstrate that two counters created by two separate calls to `make_counter()` maintain independent state.
3. Explain why the internal count is not directly accessible from outside the returned function, i.e. why it counts as 'private' state.

MEDIUM

Forwarding Arguments With args kwargs

You want a `log_call` decorator that records every call made to a wrapped function, including its positional and keyword arguments, without needing to know in advance what arguments that function takes.

```
def log_call(func):
    # TODO: implement, recording calls in wrapper.calls
    pass

@log_call
def add(a, b, c=0):
    return a + b + c
```

Your task:

1. Implement `log_call` so the decorated function still behaves normally, but `wrapper.calls` accumulates a list of (args, kwargs) tuples, one per call, in the order they were made.
2. Explain why `*args` and `**kwargs` are necessary here instead of writing `wrapper(a, b, c)` explicitly.
3. Demonstrate that `add(1, 2)` followed by `add(1, 2, c=3)` results in `wrapper.calls` containing both call records with the correct args/kwargs split.

MEDIUM

Filtering Dict Comprehension

You need a one-line utility that, given a list of words and a minimum length, builds a dictionary mapping each sufficiently long word, uppercased, to its length, filtering and transforming in a single comprehension.

```
def long_word_lengths(words, min_len):
    # TODO: implement using a single dict comprehension
    pass
```

Your task:

1. Implement `long_word_lengths(words, min_len)` as a single dict comprehension that includes only words with `len(word) > min_len`, using the uppercased word as the key and its length as the value.
2. Demonstrate `long_word_lengths(['cat', 'elephant', 'dog', 'giraffe'], 3)`, and state the exact dictionary returned.

PYTHON

MEDIUM

Custom Exception With Try Except Else Finally

You need a `parse_positive_int` function that converts a raw string into a positive integer, raising a clear, custom error for both non-numeric input and non-positive numbers, while always logging that a parse attempt completed.

```
class ConfigError(Exception):
    pass

def parse_positive_int(raw):
    # TODO: implement using try/except/else/finally
    pass
```

Your task:

1. Implement `parse_positive_int(raw)` so that: a `ValueError` from `int(raw)` is caught and re-raised as a `ConfigError` with a descriptive message; a successfully parsed but non-positive value also raises a `ConfigError`; and in all cases, success or failure, a local log list records that an attempt finished, using a `finally` block.
2. Explain why the value `<= 0` check belongs in the `else` clause of the `try` statement rather than directly after the `int(raw)` call inside the `try` block.

MEDIUM

Operator Overloading With Dunder Methods

You need a small `Vector2D` class representing a 2D vector that supports addition with `+`, equality comparison with `==`, and a readable string representation when printed or shown in a `repr` context.

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # TODO: implement __add__, __eq__, __repr__
```

Your task:

1. Implement `__add__` so that `v1 + v2` returns a new `Vector2D` whose `x` and `y` are the componentwise sums.
2. Implement `__eq__` so that two `Vector2D` instances compare equal exactly when their `x` and `y` both match.
3. Implement `__repr__` so that printing or `repr()`-ing a `Vector2D` produces a readable string like `Vector2D(4, 6)`.

